

Generalizing Manipulation Skills with a Local Coding Agent

Raman Talwar^{1*}, Elias Nijs^{1*}, Andreas Verleysen¹ and Francis Wyffels¹

Abstract—Today, progress in open-weight language models enables systems capable of writing, executing and debugging code while still running on a single workstation. Most language-driven robots give the model a fixed action interface or a trained policy. Generalizing to a new task therefore means more engineering effort or more data collection, both time-consuming. We investigate whether a local open-weight vision-language model can control a robot and one-shot generalize to new variations of a task without new human programming or training. We let a local open-weight VLM, Qwen3.8-27B, drive a UR3e robotic arm from a coding-agent harness. It writes and runs its own code above a service that implements kinematics, safety limits and classic computer vision techniques. We investigate if this system is capable of generalizing to unseen tasks. Specifically, we test it on nine tasks built from children’s toys designed to probe generalization capability across various object characteristics: color, size, shape, and task variation of those. With five trials for each task, we observe generalization in 30 out of 45 trials with durations ranging from 3.4 to 67.5 minutes depending on task complexity. We further test if there is a speedup when an agent is asked to redo the task after successful completion. This resulted in a 50% reduction in duration, indicating that there is self-improvement over time. Finally, we expose the limitations of a local coding agent. We believe that solving those limitations combined with further investigation of self-improvement over time points at a direct path toward real-world deployment of a local coding agent.

I. INTRODUCTION

Instructions in natural language are most of the time ambiguous for robots. *Stack the smaller cup on the bigger one* does not say where either cup is, how wide to open the gripper, how far down to close it, or which pixels in an image belong to the cup. Somebody has to supply those numbers, and they depend on the object in front of the robot. Most language-driven robots have a programmer supply them in advance: measure once, freeze the constants into functions such as `find(object)` and `pick(object)`, and let the model choose functions and fill in arguments [1]–[4]. New objects therefore need new programmer-supplied code and measurements. This limits the flexibility and generalization capability of the system.

Code as Policies [3] had a large language model write that code itself. However, the large language models of 2022 were weak programmers. Furthermore, their system was open-loop, meaning the agent could not debug its code. Both limits have since lifted. Open-weight models at hundreds of billions of parameters (GLM 5.3¹, Kimi K3²) and even local models (Qwen3.8-27B³) now write and debug code. Furthermore,

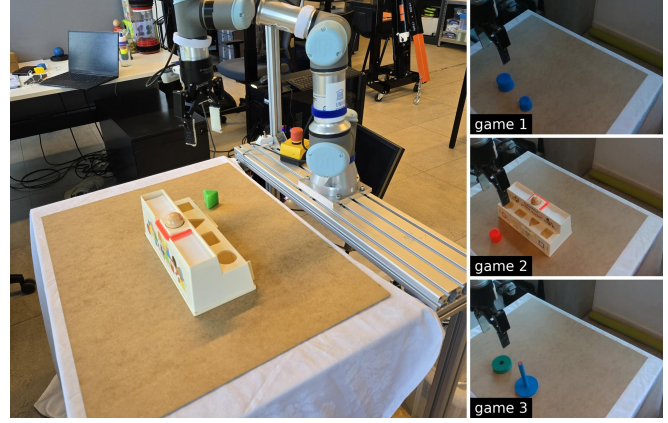


Fig. 1: The robotic arm, here with the *Sorter* game. Right: the basic tasks for which skills were provided (two blue cups, a red cylinder, the teal ring).

agent harnesses give a model a shell, a filesystem, a loop that hands tracebacks back to it [5], [6], and load long procedural documents (skills) only when they are needed.⁴

Recent work [7]–[9] puts coding agents on robots and investigates their manipulation performance. However, none of this work tests generalization capabilities on real-world hardware with a locally running open-weight model.

In this work we investigate if a local model can generalize to new object-related variations of a task. In our system, a locally served Qwen3.8-27B within a coding-agent harness operates a UR3e arm with a parallel gripper and a wrist camera (Fig. 1). A service owns the hardware and exposes nine low-level operations (Section III). Its task knowledge consists of two *skills* per game: procedural documents, each describing the task on one object instance.

We test our system on nine tabletop tasks built from children’s toys designed to probe generalization capability across object characteristics: color, size, shape, and task variation. We further test if a speedup is present when the agent is asked to complete the task a second time with a rearranged scene.

Supplementary materials, including a video of the trials, additional experimental data, agent session logs, scene-camera recordings, the system prompt, and skill documents, are available on the project page.⁵

Our contributions are:

⁴<https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.

⁵<https://rtalwar2.github.io/agent-coding-for-robot-manipulation/>

. Video: <https://youtu.be/09Z0gJm21E4>.

*Equal contribution. ¹IDLab-AIRO, Ghent University – imec, Belgium.

¹<https://huggingface.co/zai-org/GLM-5.3>.

²<https://github.com/MoonshotAI/Kimi-K3>.

³<https://huggingface.co/Qwen/Qwen3.8-27B>.

- An investigation into the generalization capability and limitations of local coding agents serving as manipulation controllers on physical hardware, including timings.
- A framework that lets a local coding agent control a robot by writing code or executing tool calls on top of a service implementing basic motion and computer vision primitives, starting from a skill. Our framework further enforces safety by limiting tool speed, gripper force and workspace bounds.
- A protocol for assessing generalization capability by using toys used in developmental psychology.
- Evidence that a coding agent improves across passes: resuming its own session on a rearranged scene halves the tool calls and cuts output tokens to a third.

II. RELATED WORK

Language models have been used for a long time as controllers for robots. The most common arrangement asks the model for a plan over a fixed inventory of skills [2]: the model proposes named actions, an affordance model scores their feasibility [1], and textual feedback from the scene closes the loop [10]. A second line asks for a program against an API of perception and control functions [3], [4], [11]. This gains loops, conditionals and arithmetic on coordinates, but the API functions are task-level and perception is provided. Later work lets a model write new low-level primitives from curated documentation [12], emit keypoint constraints per task instance with perception from large vision models [13], or control real arms zero-shot through code [14]. Constrained variants guarantee the feasibility of the generated program [15]. Frontier API-served models are now also benchmarked as direct robot controllers.⁶ End-to-end policies [16], [17] avoid the vocabulary problem at the cost of a demonstration corpus.

New developments in large language models and agentic coding led to the development of harnesses. These give a model a shell, a filesystem and an execution loop. Yang et al. [5] showed that a model can iterate on a failing program, and scores on such benchmarks [6] rose sharply as models improved. Skill libraries that grow inside such loops exist in simulation, as executable code with self-verification [18] and as reward code refined against a simulator [19]. Two recent preprints put coding agents on manipulation. ALRM [8] compares code-as-policy against tool-as-policy across models in simulation. CaP-X [7] benchmarks coding agents across interface-abstraction tiers on simulated and real arms and finds that a higher-level interface helps small models most. ASPIRE [9] has an API model discover text skills from its own failures in simulation and reports with- and without-skill trials on a real bimanual arm. Outside robotics, SkillsBench [20] reports that curated procedural skills lift smaller models towards larger ones and that a few focused skills outperform exhaustive bundles.

One-shot and few-shot imitation conditions a policy on one or two demonstration trajectories [21], [22]. Demo2Code

compiles a demonstration into code [23], and TidyBot generalizes a handful of user examples into preferences on physical hardware [24]. Learned policies generalize to unseen objects and scenes when trained on intermediate representations such as keypoints, measured over 2,000 real rollouts [25]. Text has also carried task knowledge. DROC [26] accumulates language corrections into a knowledge base for a code-writing executor, and AutoManual [27] has an agent compile its own interaction into a rule manual that guides smaller models in simulation. ELLMER [28] retrieves curated text for a model on a real kitchen manipulator.

Building on these coding-agent approaches, we investigate generalization across object and task variations within each game while holding the model, low-level interface and skill documents fixed. Our agent uses a locally deployed 27B vision-language model on a single workstation and adapts procedures documented for one object instance without additional human programming or model training. The agent writes additional perception and control code. We evaluate the capabilities and limitations of this configuration on physical hardware.

III. METHOD

The system has three parts (Fig. 2). The *platform* owns the robot and enforces its motion limits. It holds everything whose correctness must not depend on the model. The *agent setup* is the model, the system prompt, the tools, the skills and a workspace template with the platform manual, configured once per robot. In each *session* the model reads, writes and runs code in an isolated workspace and cannot modify the other two. Invariant hardware functions (kinematics, limits, the geometry that turns a pixel into a coordinate) live in the platform, and knowledge from one object instance lives in a skill. Scene measurements such as object width should be remeasured by the model in the session.

A. Platform layer

The platform is one hardware-owning HTTP service that wraps airo-mono,⁷ a Python robotics library. TABLE I lists nine operations exposed by the platform. `locate_pixel` maps one pixel to a point in base-frame coordinates using the depth map and the hand-eye calibration. `locate_objects` is a color-blob detector with depth information. It takes its color thresholds as required arguments and returns its evidence (sample counts, fit residuals, warnings) with every answer. `descend_to_contact` exposes the wrist force sensor. To avoid waiting for a model response at each control step, time-sensitive feedback loops, such as stall detection and stepwise guarded descent, run within the platform as blocking operations. There is no trained perception module. The platform supplies this one generic detector, while the model measures the color thresholds on the live frame and writes the rest of the perception during the run. The only learned component is the language model.

To enforce safety, every actuation request, from a tool or from code the model wrote, passes through safety limits.

⁶Robocurve, “GPT-6 Astra on robot arms,” third-party benchmark: <https://openai.robocurve.org/gpt-6-astra/>.

⁷<https://github.com/airo-ugent/airo-mono>.

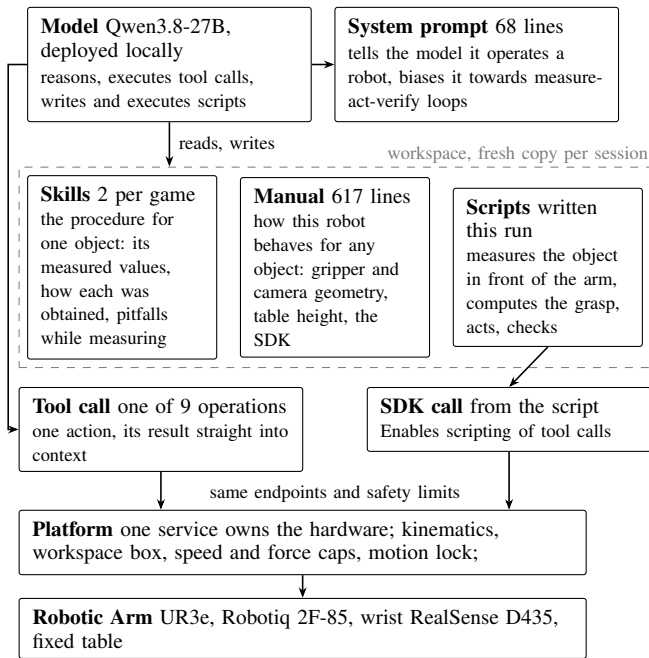


Fig. 2: Architectural overview of the framework and the interacting components.

Every Cartesian target is clamped into a workspace box. Linear speed, joint speed and gripper force are capped, and the service boots into a motion lock.

Operation	Description
look	Capture a wrist-camera frame.
get_state	Proprioception: TCP pose, joints, gripper width, the TCP force-torque reading, motion-lock state.
move_tcp	Move the tool center point to a Cartesian pose.
move_joints	Joint-space move.
grripper	Command width, force and speed.
locate_pixel	Turn one image pixel into a base-frame point via depth and the hand-eye calibration.
locate_objects	Fixed color-blob detector with depth. HSV bounds are required arguments. Returns center, top height, diameter and evidence per blob.
descend_to_contact	Step the tool down and stop on a force change, force read while stationary between steps.
set_motion	Unlock or lock the motors. The service boots senses-only.

TABLE I: The nine platform operations, available as harness tools and as a Python SDK. Both reach the same service and the same safety limits.

B. Agent setup

We use *pi*,⁸ a minimal open-source coding-agent harness, and configure the model, the system prompt, the tools, the

⁸<https://github.com/badlogic/pi-mono>.

skills and the workspace.

The system prompt tells the model it operates this robot, biases it towards measure-act-verify loops, and requires script files rather than one-liners. Its central rule: constants of the workspace box (camera offsets, table height, pad geometry) are to be trusted, while numbers measured on an object are to be re-measured.

The tools are the nine operations of TABLE I plus the harness’s file and shell tools. The model chooses to either execute a tool call or chunk tool calls via scripting using the SDK.

The skills are hand-crafted procedural documents in natural language (TABLE II). A skill holds the procedure step by step. Under each step it lists the measured values that justify it (color thresholds, grasp heights, contact forces) and how each was obtained, the verification steps, and the pitfalls met during authoring, including checks that gave a false pass. The values are marked as being valid only for the specific measured instance. The procedure is the content meant to transfer. As in the harness convention,⁴ only a skill’s name and one-line description are in the prompt. The body enters context when the task makes it relevant.

Skill	Lines	Procedure
grasp-a-cup	208	Pick one cup: measure the color window live, locate by depth evidence, touch the top, grasp at the pad band with two closes, check the lift.
stack-two-cups	192	Stack one cup on another: locate both, grasp, find the release height by force, verify from a profile view.
grasp-a-cylinder	187	Pick a cylinder: read its orientation from the silhouette, center it with or without depth, check the stall width.
insert-into-a-hole	324	Insert into the round hole: identify the hole, measure it parallax-free, find the face by touch, use a force-free descent as the alignment test, verify by look.
grasp-a-ring	182	Pick a flat ring at the jaw’s span limit: locate by color, straddle with a force-watched open-jaw descent, check the stall width.
put-a-ring-on-a-rod	277	Thread the held ring onto the rod: find the tip by its marker, predict the ground-out height, verify by a force-free descent and a top-down look.

TABLE II: The six skills, one grasp and one placement per game.

The workspace is a per-session directory copied fresh from a template, so no state crosses between tasks. The template carries the platform manual which consists of four pages describing the SDK, frames and conventions of the platform layer, the camera’s behavior, and the robot’s measured constants such as the maximum gripper width, minimum gripper width and home TCP pose.

IV. EXPERIMENTAL DESIGN

We investigate whether a local open-weight vision-language model can control a robot and one-shot generalize to new variations of a task without new human programming or training on physical hardware. Specifically, we test it on

nine tasks built from children’s toys designed to probe generalization capability across various object characteristics: color, scale, shape, and simple task variations. We measure whether it can accomplish unseen object variations of a task, how long it takes to complete those tasks and investigate the limitations of such a system.

A. Games and Tasks

In order to probe generalization capacity along object variations and task variations, we employ children’s toys. They are particularly well suited as they give compact, measurable families of related tasks: the objective stays recognizable while color, scale, shape or task changes. For the same reason, developmental psychology uses them as graded instruments of skill transfer [29], [30]. We only investigate object-related and task-related generalizations. We do not investigate generalization across other variations such as material, lighting, clutter, workspace layout, task vocabulary and robot morphology. The three games we employ are: *Cups*, a *Sorter*, and *Rings* (Fig. 3). We provide for each game, as discussed in Section III-B, two skills describing a single task from which we then test variations. With each game we perform several tasks, listed in TABLE III with their prompts, generalization axes and time caps.

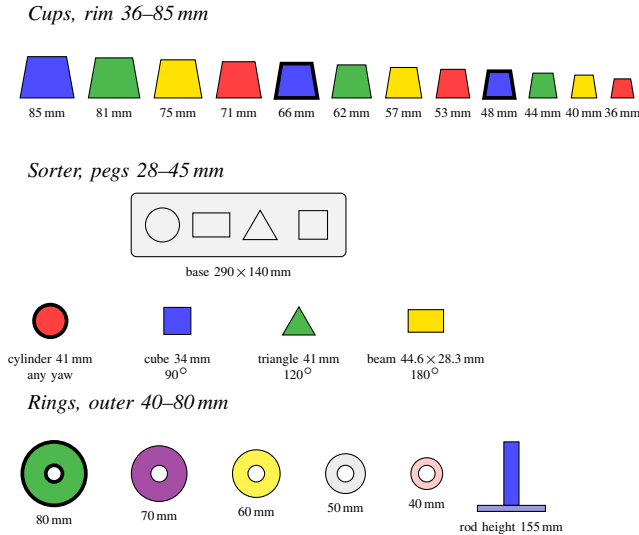


Fig. 3: Illustration of the objects involved in the games. Heavy outline indicates the object was included in a skill.

The *Cups* game uses twelve nesting cups with rim diameters ranging from 36 to 85 mm, in four repeating colors and stacked upside down. With this game we probe generalization across color, size and task variation. The basic task documented for this game in a skill is the single stacking of a small blue cup on top of a larger blue cup. From this setup we test five variations. The first asks of the system to do the task with red cups instead. The second asks to repeat the task with different-sized blue cups. The third combines the two former and asks the system to stack a small yellow cup on top of a larger green cup. The fourth combines scale

and color and introduces task variation, asking to stack three cups with varying colors and size. The fifth is a completely different task: we ask the system to order four cups in a straight line from smallest to largest.

The second game, *Sorter*, is a peg-in-hole game. It consists of a base with four different holes varying in shape and accompanying pegs which also vary in color. In order, there is a circular hole with a red cylinder, a rectangular hole with a yellow beam, a triangular hole with a green prism and a square hole with a blue cube. The pegs fit their holes with about 1 mm of clearance per side. With this game we probe shape generalization. The skill we provide for this game describes insertion of the red cylinder in the circular hole. We then ask the system to insert the other pegs as well. The difficulty within this game is ordered by yaw symmetry: the cylinder (the skill’s object) fits at any wrist angle, the cube every 90°, the triangle every 120°, the beam every 180°.

The third game, *Rings*, consists of five rings and a rod. The five rings vary in size, ranging from 40 to 80 mm, and color. The rings fit on the rod with a clearance of 2 mm. With this game we ask the system to perform a single task: stack the rings in order on the rod. This task combines color, size and task variation. The skill we provided upfront documents how to grab the largest, teal ring and insert it on the rod.

For each task we define a time cap upfront so no experiment can last indefinitely. This time cap is derived from the basic tasks documented in the skills multiplied by four to account for the added complexity from generalization, and again multiplied by the number of objects the robot has to pick up.

B. Setup

The robotic arm is a UR3e arm on a fixed table equipped with a Robotiq 2F-85 parallel gripper with the tool center point at the fingertip pinch, and an Intel RealSense D435 color-depth camera on the wrist, eye-in-hand calibrated.

The model we employ is Qwen3.8-27B,³ a dense 27B vision-language model, served with vLLM 0.27.1 (FP8 weights and KV cache, speculative decoding with three draft tokens) on the workstation that also hosts the platform (NVIDIA RTX PRO 6000, 96 GB). The context window is 262,144 tokens. Sampling uses the model’s default inference parameters (temperature 1.0, top- p 0.95, top- k 20) and reasoning runs at effort *low*. Reasoning is the process of generating thinking steps before producing a final answer.

C. Scoring and interventions

We perform five trials for each task, from now on abbreviated as $t1$ – $t5$, and record the following quantities: the success or failure, the duration, the tokens used, the number of tool calls and the percentage of tokens dedicated to reasoning. In the ideal case the agent completes the task without problems. However, sometimes an intervention is necessary. We distinguish between three interventions: small intervention, emergency stop and protective stop. A small intervention happens when a mistake by the agent results in an object lying sideways or when an object falls from the

Tasks	Prompt	On the table	Variation Axes				Time Cap
			Color	Size	Shape	Task Var.	
Cups: Red Cups	Stack the smaller red cup on top of the larger red cup.	medium red, large red	•	•			22 min
Cups: Large blue cups	Stack the small blue cup on top of the large blue cup.	small blue, large blue		•			22 min
Cups: Yellow and Green Cups	Stack the yellow cup on top of the green cup.	small yellow, large green	•	•			22 min
Cups: Three Cups	Stack the three cups on the table into one tower, largest at the bottom.	large yellow, medium green, small green	•	•		•	66 min
Cups: Sort Cups	Arrange the four cups on the table in a straight line, from smallest to largest.	large green, medium yellow, small blue, small red	•	•		•	88 min
Sorter: Beam	Put the yellow rectangular block into its matching hole in the shape sorter.	sorter, yellow beam	•		•		68 min
Sorter: Prism	Put the green triangle block into its matching hole in the shape sorter.	sorter, green triangle	•		•		68 min
Sorter: Cube	Put the blue cube into its matching hole in the shape sorter.	sorter, blue cube	•		•		68 min
Rings	Stack all the rings on the table onto the rod to rebuild the tower, largest at the bottom.	rod, base, all five rings	•	•		•	316 min

TABLE III: The four generalization axes from the base task for each new task, with each task’s prompt and time cap. A bullet marks an axis the task varies relative to the instance its skill documents.

table. When that happens, we put the object upright and back on the table, respectively. Furthermore, a protective stop or emergency stop intervention is allowed for safety reasons. A protective stop is triggered by the robotic arm while the emergency stop is triggered by the operator.

D. Second Pass

We further test if there is a speedup when the agent is asked to do a second pass after successful completion of the task. In order to evaluate this we resume the sessions of the fastest and slowest successful trials of each task and ask the agent to perform the task again. The prompt we use for this is the following:

“That worked - the task was completed correctly. The scene has been reset and the objects are in new positions. Do the task again: *<original task prompt>*”

We measure the same quantities as in the first passes of each task in order to determine if there is a possible improvement.

V. RESULTS

We investigated if a local open-weight vision-language model can control a robotic arm and generalize to new variations of a task. In order to evaluate this, we tested such a model, Qwen3.8-27B, on three children’s games capable of probing generalization across various axes (TABLE III). We derived nine tasks from the three base tasks documented in the skills, and ran each five times. TABLE IV reports the per-task results.

A. General Behavior

In general the grid took 15.8 robot-hours, produced 3.18 million output tokens, resulted in 2,043 tool calls and executed 1,034 motion commands. Out of all 45 trials 30

were successful without any intervention. Across all trials the agent followed a similar pattern. It first read the skill and relevant manual pages. Then it measured the scene using self-authored perception scripts and proceeded with writing pick-and-place scripts and executing them. Finally, it verified what it did by looking from a predefined pose. In case of an error, it looped back to measuring the scene. Reasoning accounted for 71% to 85% of output tokens in the task-level medians, with reasoning share increasing with task complexity (*Sorter*), therefore increasing task duration. This is partially caused by our identity file which biased the model toward chunking actions, and partially by enabling reasoning within the model. Empirically we observed that this mode of operation caused a lower sense-think-act frequency and more deliberate action, resulting in a higher success rate and faster task completion. The longest thinking turns happened following a surprise such as a jam or a contradiction between measurements. Trials containing them took longer in general and resulted in outlier trial durations. This occurred most in game two and game three. Out of all tasks, 30 ended without any intervention, six required a small intervention, one an emergency stop and we encountered seven protective stops.

The biggest successes can be attributed to two things. First, the skills document tasks as formulas and procedures with certain values. The agent successfully used these and re-measured values when necessary. Second, verification behavior caught failures when they happened. For example, a stall width that did not hold after the lift proved a grasp failed. The agent then knew to try again.

Task	Success (no interv.)	Duration (min)			Timeouts (#)	Tokens (Median)	Reas. (%)	Interventions (#)		
		Min	Med	Max				Small	E-stop	Prot. stop
Cups: Red Cups	4/5	3.8	4.4	22.0	1	14.7k	71	1	–	–
Cups: Large blue cups	5/5	3.4	6.1	6.7	–	23.5k	76	–	–	–
Cups: Yellow and Green Cups	5/5	3.8	7.2	9.9	–	30.8k	74	–	–	–
Cups: Three Cups	3/5	8.0	12.7	14.6	–	40.5k	77	1	–	1
Cups: Sort Cups	3/5	12.0	13.8	15.8	–	43.3k	80	1	–	1
Sorter: Cube	4/5	12.9	28.8	60.8	–	105.1k	82	–	–	1
Sorter: Prism	1/5	11.8	32.4	67.2	1	111.2k	85	1	1	2
Sorter: Beam	2/5	38.4	42.6	67.5	–	150.2k	82	2	–	1
Rings	3/5	22.8	29.8	59.0	–	99.1k	73	–	–	1

TABLE IV: Outcomes of the nine tasks at $N=5$. *Success* counts only trials that finished without any operator intervention. Durations are wall-clock minutes. *Timeouts* are trials stopped at the task’s time cap. *Tokens* is the median output tokens per trial and *Reas.* the median share of those spent on reasoning. Interventions are counted per category as defined in section IV. Fig. 4 plots the durations.

B. Failure Modes

We encountered three failure modes: perception failures, anchoring on faulty assumptions, and unawareness of the robot’s morphology and the direct space around it.

The first category of failures consists of perception failures, a wrong reading of the scene. In *Sorter: Cube* t4 the agent named the rectangular opening the square one and kept attempting to put it in this hole with increasing force, until the arm triggered a protective stop. In *Sorter: Prism* t1 and *Sorter: Prism* t3 it read the standing prism as lying on its side and attempted to grasp it from an incorrect approach angle leading to an emergency stop to prevent self-collision. In *Rings* t1 it measured the pink ring larger than the white one from an oblique view and built the tower in the wrong order. In *Rings* t2 the white-ring detector returned the yellow ring and the arm descended onto the rod, pressing the rod into the table, leading to a protective stop. We believe this category of errors can be attributed to the limitations of classical computer vision and can possibly be resolved by using more advanced deep learning methods.

The second category consists of failures where the agent anchors itself on a faulty assumption. Although the agent recovered from a faulty assumption in 10 out of 30 successful trials, in four runs it did not, leading to a failure. In *Sorter: Cube* t4 it measured the clearance nine different ways but never asked whether it had the right hole. In *Sorter: Prism* t2 it swept either the wrist angle or the position for forty minutes but never swept them together. This led to either the position being correct but not the angle or the angle being correct but not the position. In *Cups: Red Cups* t5 the agent performed a measurement from a pose from which it couldn’t be made. It realized the measurement was wrong but proceeded to remeasure from the faulty pose. In *Sorter: Beam* t4 it thought the square hole was the rectangular hole and never realized this was wrong. This led to an outlier duration and ended with the agent descending into the sorter and triggering a protective stop. We think this category of errors is mostly intrinsic to the model we used.

The third category consists of failures caused by an

unawareness of morphology and the direct space around it. The agent has no body schema, and the platform’s checks cover only the tool center point. In *Sorter: Prism* t1 and *Sorter: Prism* t3 it rotated the tool ninety degrees in mid-air. The point stayed inside the workspace box while the gripper body swept toward the arm in one case and into the table in the other. This led to an emergency stop and a protective stop, respectively. In *Cups: Three Cups* t5 the agent descended onto a cup with the jaws closed instead of open, in *Sorter: Beam* t4 and *Sorter: Prism* t4 onto the sorter with the jaws open, and in *Cups: Sort Cups* t5 a fingertip landed on the cup it had placed a moment earlier. In all cases, the agent pressed an object into the table leading to a protective stop. We think this can be resolved by handing the agent a simulation environment which includes its own morphology and possibly the scene. A different solution could be including a scene camera so the agent can see itself.

The task the agent did worst on is the *Sorter: Prism*. There was no single leading cause but rather a combination of all the above categories.

C. Second Pass Results

A second pass on an already solved task was faster and cheaper across the board (TABLE V, Fig. 4). Over the 17 resumed trials, the second pass completed the task two times faster (time ratio 0.48). The agent used about half the tool calls of the first pass (354 against 734, ratio 0.48) and about a third of the output tokens (ratio 0.34). The output tokens decreased proportionally more than tool calls, while the median reasoning share of output tokens fell from 80% to 67%. This is consistent with reduced reasoning demands when reusing prior work. The agent kept the conclusions from the first pass, such as the color windows, the object heights, the identity of the matching hole and the wrist angle that worked, and spent its remaining effort on re-locating the objects in the new layout. For the cups this brought every task from four to fourteen minutes down to two to eight minutes. For the sorter and the rings the gains were larger in absolute terms but less uniform, and the second pass did

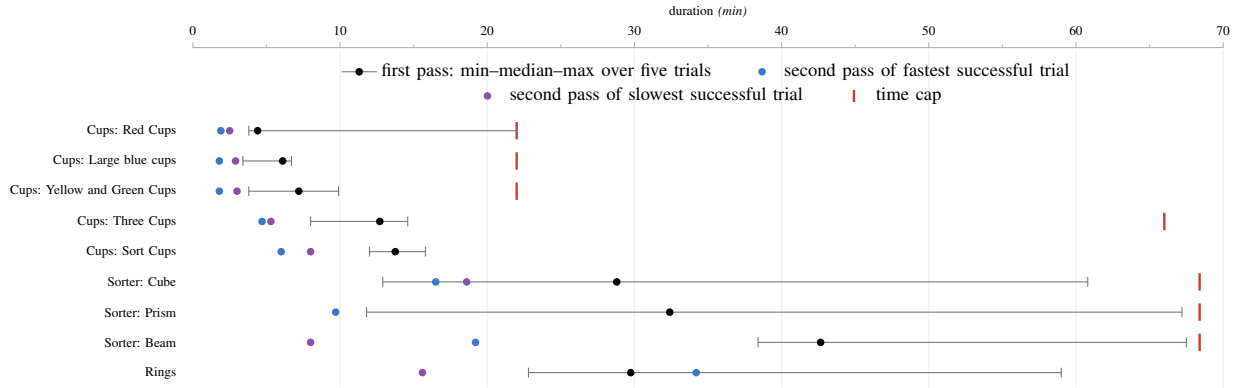


Fig. 4: Per-task duration across the $N=5$ first-pass trials: the bar spans min to max and the black dot marks the median. Blue and purple dots mark the second-pass durations, each run by resuming an already-successful trial of that task: blue resumes the fastest first-pass trial, purple the slowest (the prism has a single successful trial, drawn blue). The red tick marks the task’s time cap; the caps for Sort Cups (88 min) and the Rings (316 min) fall beyond the axis.

not remove the first-pass failure modes: one resumed beam trial ended in a protective stop, and one resumed ring trial took longer than its first pass because the agent failed to grasp the first ring five times before the rest went quickly. The resumed trial’s rank transferred. In six of the eight tasks with two second passes, the resume of the faster first-pass trial was again the faster one. What carries over is therefore not only the set of measured facts but also the quality of the plan the first pass arrived at: a first pass that found a clean path re-executes it cleanly, while one that reached its result by a detour tends to repeat part of the detour. There were two exceptions where the second pass hit a failure mode and took longer (Fig. 4). Sixteen of the seventeen second passes completed without intervention.

These gains should be interpreted as in session reuse rather than be seen as evidence of persistent learning. Resumed sessions retain conversation history, measurements and generated scripts, and our experiment does not isolate their individual contributions to the speedup. Nevertheless, the improvement on rearranged scenes is promising: the agent can reuse prior work without repeating the full discovery process. This motivates future experiments on whether that experience can be consolidated into skills that benefit fresh sessions.

VI. CONCLUSION

We asked whether a local open-weight vision-language model can control a robot and generalize, without new human programming or training, to variations of a task it has only read about. Building on recent work that puts coding agents on robots, we held the interface, the model and the skill text fixed and varied the objects and task requirements withing each game, on physical hardware. Qwen3.8-27B completed 30 of 45 trials without intervention. It generalized across color and size in 20 of 25 cup trials, inserted every unseen peg at least once, and rebuilt the ring tower in three of five trials. Where prior benchmarks vary the interface or the model and lean on API-served frontier models [7]–[9],

Trial	Tool calls			Tokens		
	1st	2nd	Ratio	1st	2nd	Ratio
Cups: Red Cups t2	24	12	0.50	23.5k	6.0k	0.26
Cups: Red Cups t3	18	9	0.50	12.6k	4.0k	0.32
Cups: Large blue cups t3	11	4	0.36	13.0k	3.5k	0.27
Cups: Large blue cups t5	23	13	0.57	28.6k	9.1k	0.32
Cups: Yellow and Green Cups t2	16	8	0.50	13.8k	3.6k	0.26
Cups: Yellow and Green Cups t5	24	14	0.58	42.5k	7.3k	0.17
Cups: Three Cups t1	22	14	0.64	27.9k	11.4k	0.41
Cups: Three Cups t3	30	18	0.60	59.7k	13.8k	0.23
Cups: Sort Cups t2	24	9	0.38	39.8k	9.9k	0.25
Cups: Sort Cups t3	31	17	0.55	59.7k	15.2k	0.26
Sorter: Cube t3	105	32	0.30	145.2k	25.7k	0.18
Sorter: Cube t5	29	15	0.52	57.2k	23.5k	0.41
Sorter: Prism t5	85	32	0.38	111.2k	21.4k	0.19
Sorter: Beam t1	88	22	0.25	108.2k	25.0k	0.23
Sorter: Beam t5	70	43	0.61	109.3k	58.3k	0.53
Rings t3	51	55	1.08	83.8k	86.6k	1.03
Rings t5	83	37	0.45	118.0k	29.6k	0.25
All 17	734	354	0.48	1054.1k	354.0k	0.34

TABLE V: The 17 second passes, each a resumed (fastest/slowest) first-pass trial of the task. Tool calls and output tokens for the first and second pass. *Ratio* is the second-pass value over the first-pass value. Totals over all 17 in the last row.

we measure generalization across object and task variations within each game, using a 27B model that runs on one workstation and can write its own perception code using classical computer vision.

In our experiments we encountered three failure modes. First, the agent misreads the scene from its single wrist camera, taking the square hole for the rectangular one or a standing prism for a lying one. Second, it anchors itself on a wrong reading and re-measures instead of re-questioning its assumptions. This can turns a wrong label into an hour-long trial. Third, it lacks awareness of its own morphology: the platform guards only the tool center point, so a rotated tool or a wrongly opened gripper can pass every check and end in a protective stop. Eight of the eleven failures ended in such a stop, one in an emergency stop. Furthermore, the

system is not yet fast enough. A cup task takes four to fourteen minutes, a sorter task half an hour to an hour, and 80% of the models output tokens is reasoning, not the robot moving. The time is bimodal per task: a trial that gets its first placement right runs in minutes, one that does not spends its remaining budget in the re-measure loop. A second pass on a solved task cuts tool calls in half and output tokens to a third, furthermore the fast trials stay fast on resume. When deploying a local coding agent, it is therefore beneficial to have a pre-deployment period where the agent learns the task.

In general, a local coding agent shows promise for real-world applications. It successfully generalized over several object characteristics and task variations. However, bridging the gap consists of solving the identified failure modes and making the system faster. We believe there are two interesting next steps. The first is adding sensory information to remedy both perception and morphology awareness. The second is further exploring the presence of a learning and skill consolidation phase.

ACKNOWLEDGMENT

Parts of the manuscript text were drafted and revised with Claude Code (Fable 5.1, Opus 5.1). It was also used in drafting the skills for the framework and editing code. All AI usage happened under the authors' direction. The authors checked every stated number and claim.

This research was partially funded by the Flanders AI Research Program.

REFERENCES

- [1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *Conference on Robot Learning (CoRL)*, 2022.
- [2] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *International Conference on Machine Learning (ICML)*, 2022.
- [3] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500.
- [4] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "ProgPrompt: Generating situated robot task plans using large language models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [5] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations (ICLR)*, 2024.
- [7] L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang *et al.*, "CaP-X: A framework for benchmarking and improving coding agents for robot manipulation," in *International Conference on Machine Learning (ICML)*, 2026, concurrent work.
- [8] V. Gaboardi dos Santos, I. Khadraoui, I. Farhat *et al.*, "ALRM: Agentic LLM for robotic manipulation," arXiv:2601.19510, 2026, preprint; concurrent work.
- [9] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, M. Chowdhury, Y. Zhu, L. Fan, and G. Wang, "ASPIRE: Agentic skills discovery for robotics," arXiv:2607.00272, 2026, preprint, June 2026; verified against the arXiv record 2026-09-07.
- [10] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng *et al.*, "Inner monologue: Embodied reasoning through planning with language models," in *Conference on Robot Learning (CoRL)*, 2022.
- [11] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3D value maps for robotic manipulation with language models," in *Conference on Robot Learning (CoRL)*, 2023.
- [12] M. Gonzalez Arenas, T. Xiao, I. Singh, V. Jain *et al.*, "How to prompt your robot: A PromptBook for manipulation skills with Code as Policies," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [13] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, "ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation," in *Conference on Robot Learning (CoRL)*, 2024, pp. 4573–4602.
- [14] Gemini Robotics Team, "Gemini robotics: Bringing AI into the physical world," arXiv:2503.20020, 2025.
- [15] A. Curtis, N. Kumar, J. Cao, T. Lozano-Pérez, and L. P. Kaelbling, "Trust the PRoC3S: Solving long-horizon robotics problems with LLMs and constraint satisfaction," in *Conference on Robot Learning (CoRL)*, 2024.
- [16] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning (CoRL)*, 2023.
- [17] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn *et al.*, " π_0 : A vision-language-action flow model for general robot control," in *Robotics: Science and Systems (RSS)*, 2025.
- [18] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research (TMLR)*, 2024.
- [19] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *International Conference on Learning Representations (ICLR)*, 2024.
- [20] X. Li, Y. Liu, W. Chen *et al.*, "SkillsBench: Benchmarking how well agent skills work across diverse tasks," arXiv:2602.12670, 2026.
- [21] N. Di Palo and E. Johns, "Keypoint action tokens enable in-context imitation learning in robotics," in *Robotics: Science and Systems (RSS)*, 2024.
- [22] V. Vosylius and E. Johns, "Instant policy: In-context imitation learning via graph diffusion," in *International Conference on Learning Representations (ICLR)*, 2025.
- [23] H. Wang, G. Gonzalez-Pumariaga, Y. Sharma, and S. Choudhury, "Demo2code: From summarizing demonstrations to synthesizing code via extended chain-of-thought," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [24] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, and T. Funkhouser, "Tidybot: Personalized robot assistance with large language models," *Autonomous Robots*, vol. 47, no. 8, 2023.
- [25] T. Lips, M. Moletta, M. C. Welle, D. Kragic, and F. Wyffels, "On the generalization capabilities, design choices and limitations of keypoint imitation learning," arXiv:2605.26649, 2026.
- [26] L. Zha, Y. Cui, L.-H. Lin, M. Kwon *et al.*, "Distilling and retrieving generalizable knowledge for robot manipulation via language corrections," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [27] M. Chen, Y. Li, Y. Yang, S. Yu, B. Lin, and X. He, "Automanual: Constructing instruction manuals by LLM agents via interactive environmental learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [28] R. Mon-Williams, G. Li, R. Long, W. Du, and C. G. Lucas, "Embodied large language models enable robots to complete complex tasks in unpredictable environments," *Nature Machine Intelligence*, vol. 7, pp. 592–601, 2025.
- [29] H. Örnkloo and C. von Hofsten, "Fitting objects into holes: On the development of spatial cognition skills," *Developmental Psychology*, vol. 43, no. 2, pp. 404–416, 2007.
- [30] P. M. Greenfield, K. Nelson, and E. Saltzman, "The development of rulebound strategies for manipulating seriated cups: A parallel between action and grammar," *Cognitive Psychology*, vol. 3, no. 2, pp. 291–310, 1972.